

Relazione gestione di reti

Leaky bucket integrato nel bridge bidirezionale



UNIVERSITÀ DI PISA

Damiano Ottaviano
d.ottaviano@studenti.unipi.it
671944

Anno Accademico 2025/2026

1 Introduzione

- Il progetto si basa sull'integrazione del leaky bucket, per limitare il traffico eccessivo dei dati, nel bridge bidirezionale. Il progetto è stato fatto con il **linguaggio c** utilizzando la libreria **pcap** per la trasmissione e ricezione di pacchetti. Come scelte progettuali, per rendere più facile la visione dei pacchetti in transito, ho inserito un filtro manuale (senza l'uso di pcap) che legge solamente pacchetti UDP (quindi con protocollo 17) e con un certo numero identificativo per evitare di leggere pacchetti UDP esterni, con una struttura del pacchetto leggermente diversa, a quelli che inviamo con sender.c. Questo per evitare di confondersi con altri pacchetti in transito ed evitare errori, visto che la struttura del pacchetto è stata leggermente alterata.
- Nel bridge, ogni volta che arriva un pacchetto, verranno stampati sullo schermo:
 - Counter attuale dei pacchetti con il relativo IP sorgente del pacchetto.
 - Lista degli indirizzi sorgenti che sono stati catturati di recente e la lista nera.
 - Viene specificato il nome dell'interfaccia che sta lavorando in quel momento. Più in avanti si vedrà meglio con degli esempi.

2 Descrizione

Nel progetto sono presenti i seguenti file:

- mybridge.c: bridge bidirezionale
- sender.c: programma per inviare pacchetti all'interfaccia desiderata
- addresses.txt: file di testo da cui prelevare gli indirizzi ip che verranno usati dal sender.c che invia gli stessi pacchetti diversificati solamente dall'ip
- structure.h: contiene le strutture dati necessarie. Sia per la costruzione del pacchetto, e sia quelle per la costruzione di una lista concatenata.
- pcount.c: utilizzato per ricevere pacchetti e ri-inoltrarli sulla stessa interfaccia.
- utils.c: contiene la logica e l'implementazione del leaky bucket, utilizzato nel bridge.c
- utils_leaky_bucket.h: contiene le firme dei metodi implementati in utils.c

Come scelta progettuale, i pacchetti che verranno trasmessi sono di tipo UDP, aggiungendo un ulteriore struttura dati per l'header dell'udp, affiancato all'header ip. Il pacchetto è costituito anche da un header ethernet e un payload, che contiene:

- Hop: un intero che rappresenta la quantità di destinazioni raggiunte, utile per capire se il bridge deve inoltrare il pacchetto dalla prima interfaccia (evitando il loop infinito).

- Message: un messaggio che verrà letto sempre nel caso in cui l'indirizzo da cui proviene non si trova nella lista nera.

```

struct eth_header {
    uint8_t dst_mac[ETH_ALEN];
    uint8_t src_mac[ETH_ALEN];
    uint16_t ethertype;
} __attribute__((packed));

struct ip_header {
    uint8_t vl;
    uint8_t type_service;
    uint16_t total_length;
    uint16_t identification;
    uint16_t fo;
    uint8_t ttl;
    uint8_t protocol;
    uint16_t checksum;
    uint32_t src_ip;
    uint32_t dst_ip;
} __attribute__((packed));

struct udp_header {
    uint16_t src_port;
    uint16_t dst_port;
    uint16_t length;
    uint16_t checksum;
} __attribute__((packed));

struct payload {
    uint32_t hop;
    char message[32];
} __attribute__((packed));

```

- Alla fine di ogni struct viene utilizzata la direttiva `__attribute__((packed))` per evitare il padding nelle strutture e alcuni disagi per l'accesso ai campi per l'offset, ottimizzando l'uso della memoria.
- I pacchetti vengono inviati tramite `sender.c`, che permette di inviare gli stessi pacchetti, differenziati solamente dall'indirizzo ip, utile per capire l'approccio del leaky bucket nel bridge. Il sender prende come input l'interfaccia a cui mandare pacchetti (che sarà l'interfaccia di ingresso del bridge) e un file di testo contenente diversi indirizzi ip che verranno scelti, come ip sorgente del pacchetto, tramite un numero digitato sul prompt, utilizzato come indice identificativo di un certo ip all'interno dell'array contenente tutti gli indirizzi del file. Un esempio di avvio, molto semplice, del file `sender.c`:

```

Range {0-8}
1
Pacchetto inviato!
4
Pacchetto inviato!
0
Pacchetto inviato!
33
Numero inserito fuori dal range!
2
Pacchetto inviato!

```

Figure 1: Esempio di avvio del file `sender.c`

3 Come funziona il Leaky Bucket

- Vengono utilizzate due liste concatenate, dove una (chiamata **table** nel bridge di tipo **struct bucket**) serve per registrare gli indirizzi ip recenti, e l'altra (**blacklist**) per registrare gli indirizzi ip che hanno superato una certa soglia, limitando la loro analisi e l'inoltro dei loro pacchetti alla seconda interfaccia per un certo periodo di tempo.
- Ad ogni indirizzo presente nella lista degli ip recenti (**table**), viene associato un livello inizializzato ad 1 che rappresenta la frequenza dei pacchetti che arrivano con un quell'indirizzo. Chiaramente, un indirizzo non rimarrà all'infinito nella lista, per cui ho creato una costante che indica il tempo di vita rimasto per un livello che è uguale per ogni indirizzo. L'idea è che ad ogni nuovo pacchetto, viene fatta una revisione in **table**, e se c'è un indirizzo che era lì da un certo periodo di tempo, il livello si può decrementare di un certo numero che dipende da quante volte rientra il valore della costante TTL da quanto tempo era passato dall'ultimo pacchetto. Se non arriveranno più pacchetti da quell'indirizzo dopo un po' di tempo, il suo livello può arrivare a 0, per poi eliminare quell'ip dalla **table**.
- Quando arriva un pacchetto, se l'indirizzo ip relativo a quel pacchetto è presente in **table**, il livello verrà incrementato di 1. Se il livello dovesse superare il limite, allora l'ip relativo a quel livello verrà inserito nella lista nera per un certo periodo di tempo.
- Macro importanti in `utils_leaky_bucket`:
 - `CAPACITY`: è la soglia massima. Se viene superato, l'indirizzo viene inserito nella **blacklist**.
 - `TTL_IP_IN_BUCKET`: tempo rimanente dell'indirizzo nella lista principale degli indirizzi che hanno inviato un pacchetto recentemente.
 - `TTL_IP_IN_BLACKLIST`: tempo rimanente dell'indirizzo nella **blacklist**.

```
struct Bucket {
    uint32_t ip;
    unsigned int level;
    time_t timestamp;
    struct Bucket* next;
};

struct Address_node {
    uint32_t ip;
    time_t timestamp;
    struct Address_node* next;
};
```

Figure 2: Liste concatenate utilizzate in `bridge.c`

- Primo esempio di simulazione

```
damdev00@damdev-ubuntu:~/Desktop/university/gestione_reti/proje
it$ sudo ./sender -i enp0s3 -f addresses.txt -d 12345
[sudo] password for damdev00:
Range {0-8}
1
Pacchetto inviato!
2
Pacchetto inviato!
damdev00@damdev-ubuntu:~/Desktop/university/gestione_reti/proje
damdev00@damdev-ubuntu:~/Desktop/university/gestione_reti/proje$ sudo ./mybridge -
i enp0s3 -o enp0s8 -d 12345
[sudo] password for damdev00:
Capturing from enp0s3
user changed to nobody
[enp0s3] pacchetto arrivato!
[enp0s3] pacchetto inviato all'interfaccia [enp0s8]!

=====
Packet #1
IP: 192.168.1.191
=====

Bucket[2] -> 192.168.1.191

[enp0s8] pacchetto arrivato!
[enp0s8:192.168.1.191] : hello world!
[enp0s8] pacchetto inviato all'interfaccia [enp0s3]!
[enp0s3] pacchetto arrivato!
[enp0s3] pacchetto inviato all'interfaccia [enp0s8]!

=====
Packet #2
IP: 192.168.1.120
=====

Bucket[1] -> 192.168.1.120
Bucket[2] -> 192.168.1.191

[enp0s8] pacchetto arrivato!
[enp0s8:192.168.1.120] : hello world!
[enp0s8] pacchetto inviato all'interfaccia [enp0s3]!
]
```

Figure 3: Primo esempio di simulazione

- Dietro le quinte ovviamente, c'erano anche due pcount attivi per ogni interfaccia. Ogni pacchetto che arriva, viene inserito in **table** tramite un hash che calcola l'indice sommando gli ottetti dell'indirizzo. Quando il pacchetto giunge nella seconda interfaccia del bridge, viene stampato il testo del messaggio (in questo caso hello world).
- Viene sempre specificato il numero dei pacchetti che sono transitati nel bridge e l'ip sorgente del pacchetto. Inoltre, viene anche specificato da quale interfaccia arriva il pacchetto, segnalando anche l'inoltro alla [seconda—prima] interfaccia.

- Secondo esempio di simulazione

```
[enp0s8] pacchetto inviato all'interfaccia [enp0s3]!
[enp0s3] pacchetto arrivato!
[enp0s3] pacchetto inviato all'interfaccia [enp0s8]!

=====
Packet #2
IP: 192.168.1.191
=====

Bucket[2] -> 192.168.1.191

[enp0s8] pacchetto arrivato!
[enp0s8:192.168.1.191] : hello world!
[enp0s8] pacchetto inviato all'interfaccia [enp0s3]!
[enp0s3] pacchetto arrivato!
Pacchetto da 192.168.1.191 inserito nella blacklist

=====
Packet #3
IP: 192.168.1.191
=====

Bucket[2] -> 192.168.1.191

blacklist[2] -> 192.168.1.191
[enp0s3] pacchetto arrivato!
Rimosso l'indirizzo 192.168.1.191 dal bucket
[enp0s3] pacchetto inviato all'interfaccia [enp0s8]!

=====
Packet #4
IP: 192.168.1.120
=====

Bucket[1] -> 192.168.1.120

blacklist[2] -> 192.168.1.191
[enp0s8] pacchetto arrivato!
```

Figure 4: Secondo esempio di simulazione

- Qui c'è un caso di spedizione eccessiva di pacchetti da parte dell'indirizzo 192.168.1.191. In questo esempio, CAPACITY è uguale a 2. Infatti, l'indirizzo, al terzo invio, è stato inserito nella blacklist in un intervallo di tempo troppo piccolo rispetto a TTL_IP_IN_BUCKET.
- Bisogna notare che entrando nella lista nera, non è stato analizzato il contenuto del pacchetto, e quindi nessun print di hello world.

4 Istruzioni

- Indispensabile specificare due interfacce nel bridge, in entrata e in uscita, e avviare due pcount.c ognuno per ogni interfaccia. Inoltre, per ogni file.c bisogna inserire una flag che indica il numero identificativo del pacchetto che, come detto nell'introduzione, serve per evitare di leggere altri pacchetti UDP esterni non inviati con sender.c per non andare in confusione cercando di comprendere al meglio come funziona l'algoritmo introdotto.
- Compilazione:
 - Utilizzare il makefile digitando il comando **make** producendo gli eseguibili: mybridge, pcount, sender.
 - In caso di problemi con il makefile, compilare i programmi in questo modo:

```
* gcc bridge.c utils.c -o bridge -lpcap
  gcc pcount.c -o pcount -lpcap
  gcc sender.c -o sender -lpcap
```
- Esecuzione:
 - Sudo ./sender -i [interface 1] -f [file_indirizzi.txt] -d [packet_id]
Sudo ./bridge -i [interface 1] -o [interface 2] -d [packet_id]
Sudo ./pcount -i [interface1] -d [packet_id]
Sudo ./pcount -i [interface2] -d [packet_id]
 - * Esempio:

```
* sudo ./pcount -i enp0s3 -d 12345
* sudo ./pcount -i enp0s8 -d 12345
* sudo ./mybridge -i enp0s3 -o enp0s8 -d 12345
* sudo ./sender -i enp0s3 -f addresses.txt -d 12345
```